# Numbering Systems

Different numbering systems (different bases) are used as they are more useful when working in digital circuits.

In general for a base-n system,

to convert from base n to base 10 (decimal):

$$...abcd_n = (... + an^3 + bn^2 + cn^1 + dn^0)_{10}$$

In general, the maximum value for k numerals in decimal for base n is

$$n^k - 1$$

To convert from base 10 to base n, we use repeated division.

For example,

$$\begin{gathered}(an^3 + bn^2 + cn^1 + dn^0) \div n = an^2 + bn + c + d/n \Rightarrow \text{remainder d} \\ (an^2 + bn + c) \div n = an + b + c/n \Rightarrow \text{remainder c} \\ (an + b) \div n = a + b/n \Rightarrow \text{remainder b} \\ a \div n = a/n \Rightarrow \text{remainder a} \\ \therefore (an^3 + bn^2 + cn^1 + dn^0)_{10} = abcd_n\end{gathered}$$

the first remainder found will be the first (rightmost) place in the base n number. As it has the lowest power, it is called the LSD, lowest significant digit.

The last remainder found will be the last (leftmost) place in the base n number. As it has the highest power, it is called the MSD, most significant digit.

(remainder fuction is basically just mod(a,b) as well)

To convert a fractional number from base 10 to base n, we use repeated multiplication

$$\begin{gathered}(an^{-1} + bn^{-2} + cn^{-3} + dn^{-4}) \times n = a + bn^{-1} + cn^{-2} + dn^{-3} \Rightarrow a \text{ is overflowed} \\ (bn^{-1} + cn^{-2} + dn^{-3}) \times n = b + cn^{-1} + dn^{-2} \Rightarrow b \text{ is overflowed} \\ (cn^{-1} + dn^{-2}) \times n = c + dn^{-1} \Rightarrow c \text{ is overflowed} \\ dn^{-1}) \times n = d.0 \Rightarrow d \text{ is overflowed} \\ \therefore \left(an^{-1} + bn^{-2} + cn^{-3} + dn^{-4}\right)_{10} = 0.abcd_n\end{gathered}$$

and take the whole number portion as the “remainder”, then collect the overflowed remainders from top to bottom.

Binary

the binary system (base 2) has only 2 numbers, 0 and 1.

For example, for converting between decimal and binary:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}$$

$$\begin{gathered} 19 \div 2 = 9R1 \\ 9 \div 2 = 4R1 \\ 4 \div 2 = 2R0 \\ 2 \div 2 = 1R0 \\ 1 \div 2 = 0R1 \text{ (last quotient needs to be 0)} \\ \therefore 19_{10} = 10011_2 \end{gathered}$$

Octal

the octal system (base 8) uses 8 numbers from 0-7 inclusive.

for converting between decimal and octal:

$$243_8 = 2 \times 8^2 + 4 \times 8^1 + 3 \times 8^0 = 163_{10}$$

$$\begin{gathered} 581 \div 8 = 72R5 \\ 72 \div 8 = 9R0 \\ 9 \div 8 = 1R1 \\ 1 \div 8 = 0R1 \\ \therefore 581_10 = 1105_8 \end{gathered}$$

Sometimes, you may be asked to convert between octal and binary, as so:

$$\begin{gathered} 23_8 \rightarrow 2_8\ 5_8 \rightarrow 010_2\ 011_2 \\ \therefore 25_8 = 010011_2 = 10011_2 \end{gathered}$$

Take each digit in the octal number and convert it into a 3 digit binary number, including the 0s at the front. Combine the binary “digits” together to get the converted binary number.

$$\begin{gathered} 110100_2 \rightarrow 110_2\ 100_2 \rightarrow 6_84_8 \\ \therefore 110100_2 = 64_8 \end{gathered}$$

Similarly, to convert a binary number to an octal number, take each group of 3 and convert it to its decimal digit, then combine them as so

(working is inaccurate !!!!!!! purely shortcut based)

**OCTAL TO BINARY SHORTCUT FOR DECIMAL TO BINARY??????????????????

Hexadecimal

hexadecimal (base 16) uses 16 symbols, [0 1 2 3 4 5 6 7 8 9 A B C D E F], with letters representing 11 onwards respectively.

| A | B | C | D | E | F |
|---|---|---|---|---|---|
| 10 | 11 | 12 | 13 | 14 | 15 |

For converting between decimal and hexadecimal,

$$D8F_{16} = 13 \times 16^2 + 8 \times 16^1 + 15 \times 16^0 = 3471_{10}$$

$$\begin{gathered} 5291 \div 16 = 330R11(B) \\ 330 \div 16 = 20R10(A) \\ 20 \div 16 = 1R4 \\ 1 \div 16 = 0R1 \\ \therefore 5291_{10} = 14AB_{16} \end{gathered}$$

Similarly, Hexadecimal and Binary can be converted between each other with a similar method, however in packets of 4 binary for one hexadecimal digit (2^3=8, 2^4=16)

$$\begin{gathered} CA7_{16} \rightarrow C_{16}\ A_{16}\ 7_{16} \rightarrow 1100_2\ 1010_2\ 0111_2 \\ \therefore CA7_{16} = 110010100111_2 \end{gathered}$$

$$\begin{gathered} 101100011110_2 \rightarrow 1011_2\ 0001_2\ 1110_2 \rightarrow B_{16}\ 1_{16}\ E_{16} \\ \therefore 101100011110_2 = B1E_{16} \end{gathered}$$

Hexadecimal can also be easily converted to Octal, by first converting hexadecimal to binary, then regrouping the bits into groups of 3 to convert back to Octal. The reverse applies.

(note that if the bits cannot be cleanly split into groups of 3s, add trailing 0s at the front to account for this)

BCD

Binary Coded Decimals (BCD) are similar to both the binary and decimal systems.

However, it is not a numbering system, but a code system instead.

Essentially, a decimal number has its digits split up and each digit converted to a 4 digit binary group, before each binary group are stitched together.

$$251_{10} \rightarrow 2\ 5\ 1 \rightarrow 0010_2\ 0101_2\ 0001_2 \rightarrow 001001010001\ (\text{BCD})$$

the system can also be reversed:

$$100100100011\ (\text{BCD}) \rightarrow 1001_2\ 0010_2\ 0011_2 \rightarrow 9\ 2\ 3 \rightarrow 923$$

BCD codes are quite useful for 7 segment displays, where BCD can be separated for each 7 segment number, and the binary allows digital control for each segment of the number.

this makes it useful for digital alarm clocks, digital meters, etc.

Bytes, Nibbles and Words

A byte is equal to 8 bits. (8 binary numerals)

A nibble is equal to 4 bits (4 binary numerals)

A word is a (vague) group of bits that represents information in a digital system. Depending on the system, it may only handle one byte at a time (word size = 8 bits), or may handle 8 bytes at a time (word size = 64 bits)

Alphanumerical Codes (ASCII)

ASCII is an alphanumerical system that relates every character to a unique decimal numeral (which each can be related to its corresponding binary, octal and hexadecimal number.)

| Dec | Hex | Name | Char | Ctrl-char | Dec | Hex | Char | Dec | Hex | Char | Dec | Hex | Char |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | Null | NUL | CTRL-@ | 32 | 20 | Space | 64 | 40 | @ | 96 | 60 | ` |
| 1 | 1 | Start of heading | SOH | CTRL-A | 33 | 21 | ! | 65 | 41 | A | 97 | 61 | a |
| 2 | 2 | Start of text | STX | CTRL-B | 34 | 22 | " | 66 | 42 | B | 98 | 62 | b |
| 3 | 3 | End of text | ETX | CTRL-C | 35 | 23 | # | 67 | 43 | C | 99 | 63 | c |
| 4 | 4 | End of xmit | EOT | CTRL-D | 36 | 24 | $ | 68 | 44 | D | 100 | 64 | d |
| 5 | 5 | Enquiry | ENQ | CTRL-E | 37 | 25 | % | 69 | 45 | E | 101 | 65 | e |
| 6 | 6 | Acknowledge | ACK | CTRL-F | 38 | 26 | & | 70 | 46 | F | 102 | 66 | f |
| 7 | 7 | Bell | BEL | CTRL-G | 39 | 27 | ' | 71 | 47 | G | 103 | 67 | g |
| 8 | 8 | Backspace | BS | CTRL-H | 40 | 28 | ( | 72 | 48 | H | 104 | 68 | h |
| 9 | 9 | Horizontal tab | HT | CTRL-I | 41 | 29 | ) | 73 | 49 | I | 105 | 69 | i |
| 10 | 0A | Line feed | LF | CTRL-J | 42 | 2A | * | 74 | 4A | J | 106 | 6A | j |
| 11 | 0B | Vertical tab | VT | CTRL-K | 43 | 2B | + | 75 | 4B | K | 107 | 6B | k |
| 12 | 0C | Form feed | FF | CTRL-L | 44 | 2C | , | 76 | 4C | L | 108 | 6C | l |
| 13 | 0D | Carriage feed | CR | CTRL-M | 45 | 2D | - | 77 | 4D | M | 109 | 6D | m |
| 14 | 0E | Shift out | SO | CTRL-N | 46 | 2E | . | 78 | 4E | N | 110 | 6E | n |
| 15 | 0F | Shift in | SI | CTRL-O | 47 | 2F | / | 79 | 4F | O | 111 | 6F | o |
| 16 | 10 | Data line escape | DLE | CTRL-P | 48 | 30 | 0 | 80 | 50 | P | 112 | 70 | p |
| 17 | 11 | Device control 1 | DC1 | CTRL-Q | 49 | 31 | 1 | 81 | 51 | Q | 113 | 71 | q |
| 18 | 12 | Device control 2 | DC2 | CTRL-R | 50 | 32 | 2 | 82 | 52 | R | 114 | 72 | r |
| 19 | 13 | Device control 3 | DC3 | CTRL-S | 51 | 33 | 3 | 83 | 53 | S | 115 | 73 | s |
| 20 | 14 | Device control 4 | DC4 | CTRL-T | 52 | 34 | 4 | 84 | 54 | T | 116 | 74 | t |
| 21 | 15 | Neg acknowledge | NAK | CTRL-U | 53 | 35 | 5 | 85 | 55 | U | 117 | 75 | u |
| 22 | 16 | Synchronous idle | SYN | CTRL-V | 54 | 36 | 6 | 86 | 56 | V | 118 | 76 | v |
| 23 | 17 | End of xmit block | ETB | CTRL-W | 55 | 37 | 7 | 87 | 57 | W | 119 | 77 | w |
| 24 | 18 | Cancel | CAN | CTRL-X | 56 | 38 | 8 | 88 | 58 | X | 120 | 78 | x |
| 25 | 19 | End of medium | EM | CTRL-Y | 57 | 39 | 9 | 89 | 59 | Y | 121 | 79 | y |
| 26 | 1A | Substitute | SUB | CTRL-Z | 58 | 3A | : | 90 | 5A | Z | 122 | 7A | z |
| 27 | 1B | Escape | ESC | CTRL-[ | 59 | 3B | ; | 91 | 5B | [ | 123 | 7B | { |
| 28 | 1C | File separator | FS | CTRL-\ | 60 | 3C | < | 92 | 5C | \ | 124 | 7C | \| |
| 29 | 1D | Group separator | GS | CTRL-] | 61 | 3D | = | 93 | 5D | ] | 125 | 7D | } |
| 30 | 1E | Record separator | RS | CTRL-^ | 62 | 3E | > | 94 | 5E | ^ | 126 | 7E | ~ |
| 31 | 1F | Unit separator | US | CTRL-_ | 63 | 3F | ? | 95 | 5F | _ | 127 | 7F | DEL |

Error Detection - Parity Method

When information is transmitted, some bits may be flipped (hence causing an error)

One way to detect errors is by appending a parity bit to the string of bits, so that there are an even number of 1s.

so to transmit the number 14 (1110) with a parity bit, it will be 11110, the leftmost bit being the parity bit, while the rest is the number. Notice how the parity bit is 1, to force the number of 1s in the string to be even(this is called using an even parity bit, forcing the number of 1s to be odd is using an odd parity bit

Thus, if an error flips only 1 bit, the parity will be odd, thus showing there is an error.

some limitations is that it can only detect, not correct the error, and only works when there is an odd number of errors. An even number of errors will not be detected via this method.